STAR-D — ESP32-S3 Pin Assignments

Extracted from `Main_PCB_STAR_D.kicad_sch` by tracing wire geometry and net labels · REXUS 38 · 3 November 2026

Read this before touching the drivers

Lift-off, SOE, SODS, all three heaters and the stepper enable are not on ESP32 pins. They sit behind an **MCP23S17 SPI I/O expander** (U9) whose chip select is GPIO6.

That means `rs422_interface.c` cannot use `gpio_get_level()`, and `gpio_driver.c` cannot use `gpio_set_level()`. Both must go through an expander driver that does not exist yet, and every read of a REXUS signal costs an SPI transaction.

How the board is actually divided

```
ESP32-S3 (U1)
     |
     |-- GPIO4  STEP_M1 ------------------> A4988 M1 + M2  (ganged)
     |-- GPIO5  DIR_M1  ------------------> A4988 M1 + M2  (ganged)
     |-- GPIO3  I2C_SCL ----------+
     |-- GPIO7  I2C_SDA ----------+-------> J4  flow sensor (SLF3S, off-board)
     |-- GPIO1  LED_R / GPIO2 LED_Y
     |-- GPIO8  CS_RASP  ---------------->  ( nothing -- see page 4 )
     |-- U0TXD / U0RXD   ---------------->  ( nothing -- see page 4 )
     |
     +== SENSOR SPI BUS  GPIO12 SCK / GPIO11 MOSI / GPIO13 MISO
            |
            |-- CS GPIO6   U9  MCP23S17 I/O expander
            |                    |-- GPB0 <- LO_IN    <- optocoupler U3 <- RXSM_LO
            |                    |-- GPB1 <- SOE_IN   <- optocoupler U5 <- RXSM_SOE
            |                    |-- GPB2 <- SODS_IN  <- optocoupler U6 <- RXSM_SODS
            |                    |-- GPB3 -> HEATER1_EN -> R_G1 -> MOSFET
            |                    |-- GPB4 -> HEATER2_EN -> R_G2 -> MOSFET
            |                    |-- GPB5 -> HEATER3_EN -> R_G3 -> MOSFET
            |                    |-- GPB6 -> EN_STEPPERS -> M1.9 + M2.9
            |                    +-- INTA  -> ( nothing -- polling only )
            |
            |-- CS GPIO10  U8  BME280        (internal)
            |-- CS GPIO46  U2  BME280        (external, "_OUT")
            |-- CS GPIO21  U7  MAX31865      (PT100)
            |-- CS GPIO14  U4  LSM6DSM       (IMU)
            +-- CS GPIO45  J1  ABP pressure  (off-board, MISO only)
```

Why this matters more than it looks

One SPI bus now carries the sensors *and* every REXUS signal *and* heater control. A stuck bus — one slave holding MISO low, one CS shorted — takes out lift-off detection, thermal control and the stepper at the same time. That is a wider single point of failure than the single LO wire, and it argues for a bus-health check that latches its own error bit rather than letting each driver fail silently.

Method: the schematic stores pin coordinates and wire segments, not a netlist. Each symbol's library pins were transformed into sheet coordinates, joined through wires and junctions with a union-find, then named from the labels landing on each net. 505 pins, 323 wires, 233 nets, 56 named nets with pins attached.

Direct ESP32-S3 pins

Every net that reaches a physical ESP32-S3 pin. “Chip pin” is the package pin number in the schematic symbol, useful when probing.

GPIO	CHIP PIN	NET	GOES TO	OWNER IN SOFTWARE	STATE
GPIO1	6	LED_R	D1 cathode (red status LED)	— not written	spare
GPIO2	7	LED_Y	D2 anode (yellow status LED)	— not written	spare
GPIO3	8	I2C_SCL	J4.5 → SLF3S flow sensor, pull-up R_I2C_SCL	i2c_interface.c	strap
GPIO4	9	STEP_M1	A4988 M1.15 **and** M2.15	gpio_driver.c	ok
GPIO5	10	DIR_M1	A4988 M1.16 **and** M2.16	gpio_driver.c	ok
GPIO6	11	CS_IOEXP	MCP23S17 U9.11, pull-up R9	**missing driver**	gap
GPIO7	12	I2C_SDA	J4.2 → SLF3S flow sensor, pull-up	i2c_interface.c	ok
GPIO8	13	CS_RASP	**nothing** — no Pi connector in schematic	spi_pi_link.c	gap
GPIO9	14	GPIO_ADC	C19 (analogue input, purpose undocumented)	— not written	?
GPIO10	15	CS_BME280	U8.2 — BME280, internal	bme_driver.c	ok
GPIO11	16	SPI_MOSI	shared sensor bus — 6 slaves	spi_interface.c	ok
GPIO12	17	SPI_SCK	shared sensor bus + J1.3	spi_interface.c	ok
GPIO13	18	SPI_MISO	shared sensor bus + J1.4	spi_interface.c	ok
GPIO14	19	CS_LSM6DSM	U4.12 — IMU	imu_driver.c	ok
GPIO19	25	USB-D−	R16 → J10 USB-C	(USB-JTAG console)	fixed
GPIO20	26	USB-D+	R17 → J10 USB-C	(USB-JTAG console)	fixed
GPIO21	27	CS_MAX31865	U7.13 — PT100 front-end	pt100_driver.c	ok
GPIO33	38	ACC_int2	**nothing** — dead stub	—	free
GPIO34	39	ACC_cs	**nothing** — dead stub	—	free
GPIO35	40	ACC_mosi	**nothing** — dead stub	—	free
GPIO36	41	ACC_sck	**nothing** — dead stub	—	free
GPIO37	42	ACC_miso	**nothing** — dead stub	—	free
GPIO38	43	ACC_int1	**nothing** — dead stub	—	free
U0TXD	49	UART-TXD	**nothing** — no transceiver, no connector	uart_tcu.c	gap
U0RXD	50	UART-RXD	**nothing** — no transceiver, no connector	uart_tcu.c	gap
GPIO45	51	CS_ABP	J1.5 → ABP pressure sensor, off-board	(driver not written)	strap
GPIO46	52	CS_BME280_OUT	U2.2 — BME280, external	bme_driver.c	strap

GPIO30–35 of the symbol (pins 30–35) carry the in-package flash: SPIHD, SPIWP, SPICS0, SPICLK, SPIQ, SPID → IC2 (W25Q01JV). Never touch them from software. GPIO0 is unused, which is the right answer for a strapping pin.

Behind the expander — MCP23S17 (U9), port B

These seven lines are the ones the flight software cares most about, and not one of them is reachable with a GPIO call.

EXP. PIN	IC PIN	NET	DIR	PATH	OWNER IN SOFTWARE
GPB0	1	LO_IN	in	RXSM_LO → optocoupler U3 → pull-up R_PULL_LO	rs422_interface.c
GPB1	2	SOE_IN	in	RXSM_SOE → optocoupler U5 → pull-up R_PULL_SOE	rs422_interface.c
GPB2	3	SODS_IN	in	RXSM_SODS → optocoupler U6 → pull-up R_PULL_SODS	rs422_interface.c
GPB3	4	HEATER1_EN	out	R_G1 → Q_HEATER1 gate → J_HEATER1	gpio_driver.c
GPB4	5	HEATER2_EN	out	R_G2 → Q_HEATER2 gate → J_HEATER2	gpio_driver.c
GPB5	6	HEATER3_EN	out	R_G3 → Q_HEATER3 gate → J_HEATER3	gpio_driver.c
GPB6	7	EN_STEPPERS	out	A4988 M1.9 and M2.9 (active low ENABLE)	gpio_driver.c
INTA	20	INTA_MCP23S17	out	**not connected** — interrupt-on-change unusable	–
~CS	11	CS_IOEXP	in	ESP32 GPIO6	spi_interface.c
SCK/SI/SO	12/13/14	SPI_*	—	shared sensor bus	spi_interface.c

Three consequences for code already written

- **An expander driver is now a prerequisite.** MCP23S17 over SPI: write `IODIRB` so GPB0–2 are inputs and GPB3–6 outputs, enable `GPPUB` pull-ups on the inputs, then read `GPIOB` and write `OLATB`. Opcode byte is `0x40 | (addr<<1) | R/W`.
- **`rs422_interface.c` changes shape.** It must cache one `GPIOB` read per cycle and have all three level getters read that cache, or five debounce samples become fifteen SPI transactions. The getters stay side-effect-free; only the refresh moves.
- **Heaters-off-on-fault is no longer a single instruction.** `gpio_driver_heaters_set(false)` now needs the SPI bus to be working. Decide, and write down, what the software does when the expander write fails while the PT100 says the chamber is too hot.

Shared sensor SPI bus — who is on it

DEVICE	REF	CHIP SELECT	NOTE
MCP23S17 expander	U9	**GPIO6**	REXUS signals in, heaters and stepper enable out
BME280 internal	U8	**GPIO10**	compensation formulas still to be copied from the datasheet
BME280 external	U2	**GPIO46**	the one `health_monitor.c` stops checking after launch
MAX31865 (PT100)	U7	**GPIO21**	fault register must be read before the resistance is trusted
LSM6DSM (IMU)	U4	**GPIO14**	INT1 brought out as `INT_LSM6DSM` but not connected
ABP pressure	J1	**GPIO45**	off-board: SCK on J1.3, MISO on J1.4, CS on J1.5, no MOSI (read-only part)

What to ask hardware, in priority order

1 Is the ESP the SPI master or the slave on the Pi link?

`CS_RASP` is GPIO8 and connects to **nothing else** — there is no Raspberry Pi connector anywhere in this schematic. It is drawn as an ESP output, which is how you wire a *master* driving the Pi's chip select. We built `spi_pi_link.c` the other way round, with the ESP as slave, because Linux `spidev` is master-only.

Both cannot be true. Until this is settled, the handshake GPIO, the one-cycle pipeline and the degradation direction in that file are all unverified.

2 Where is the RS-422 link to the RXSM?

There is no RS-422 transceiver in the schematic. `UART-TXD` and `UART-RXD` leave U0TXD/U0RXD and terminate immediately. `d-sub-distribution.kicad_sch` contains the DA15 connector J2 and **zero wires**.

So the telemetry downlink has no hardware path in this revision, and the pins it would use are the ROM console UART, which the bootloader also prints on. Separately: which UART do the TCU and the RXSM each get, and on which pins?

3 GPIO45 as a chip select is a boot-time hazard

GPIO45 is the VDD_SPI strapping pin: sampled LOW at reset it selects 3.3 V for the flash, HIGH selects 1.8 V. A chip select idles HIGH. If the off-board ABP sensor on J1 presents a pull-up on its CS line, the flash is under-volted and the board will not boot. Confirm there is a pull-down on `CS_ABP` and no pull-up at the sensor end. GPIO46 (`CS_BME280_OUT`) is the ROM-message strap, defaults pulled down, so it is milder — but confirm from the S3 datasheet that it can drive an output at all, because the equivalent pin on the ESP32-S2 is input-only and that assumption travels between parts.

4 Are the two steppers meant to be ganged?

M1 and M2 share `STEP_M1`, `DIR_M1` and `EN_STEPPERS`. They can only ever move together, in the same direction, by the same number of steps. If the mechanism needs them independent, this is a board change, not a software one.

5 Smaller questions

- `INTA` on the expander is unconnected — confirm that polling LO/SOE/SODS at the control-loop rate is accepted, and that the latency is inside the SED's timing budget.
- `GPIO_ADC` (GPIO9) goes to C19 and nothing documented. What is it measuring? If it is the bus voltage, `sensors.c` already has a `voltage_mV` field waiting for it.
- Port A of the expander is unused, so eight more lines are free if a signal must move.
- Does the module carry octal PSRAM? If so GPIO33–37 must stay unused — which they currently are, the `ACC_*` nets being dead stubs.
- LED_R and LED_Y have no software owner. Worth a defined blink pattern before integration week, when it is the only diagnostic available with the experiment closed up.

Things the trace confirms are right

- The REXUS signals are optically isolated, each with its own pull-up, so they read as clean levels — the active-LOW, debounced, level-not-edge design in `rs422_interface.c` is correct for this hardware.
- Three independent heater channels, one gate resistor each, exactly as `temperature_control.c` assumes when it drives them as one zone.
- The ABP has no MOSI, correct for that part; the flow sensor is the only I²C device, which is what `i2c_interface.c` assumes. GPIO0 is unused and no signal net touches the flash pins.